

Rapport - SatElite

Ramie NASSERALDIN

Presentation du projet

SatElite est mon implementation du mini-projet de logique autour du puzzle **Easy as ABC**. Le but est de partir d'une grille de puzzle, de la transformer en formule SAT, puis d'utiliser un solveur pour retrouver une grille complete qui respecte toutes les contraintes.

J'ai choisi de developper le projet en **Java** parce que c'est un langage que j'apprécie et avec lequel je suis à l'aise. La resolution SAT n'est donc pas faite par un solveur écrit à la main : les clauses sont construites par mon programme, puis données à Sat4J.

Le projet s'appelle **SatElite**. Il contient plusieurs fichiers sources dans le dossier `src/`, des instances de puzzle dans `Puzzle/`, un dossier `Dimacs/` pour les fichiers CNF generes, un `makefile` pour automatiser les commandes et un `pom.xml` pour gerer la dependance Sat4J.

Fonctionnement general

Le programme suit le meme enchainement pour chaque puzzle.

D'abord, il lit le fichier d'entree. Ce fichier indique la variante du puzzle, les indices autour de la grille et, si elle existe, une grille deja partiellement remplie. J'ai garde un format simple afin de pouvoir ajouter facilement de nouvelles instances.

Ensuite, le programme construit les variables SAT. Une variable represente le fait qu'une case donnee contient une lettre ou un symbole donne. La conversion en numero DIMACS est faite dans la classe `SatVariables`.

Après cela, les contraintes du puzzle sont ajoutees sous forme de clauses. Ces contraintes imposent qu'une case ne contienne qu'un seul symbole, que les lettres apparaissent correctement dans les lignes et les colonnes, et que les indices extérieurs soient respectes.

Une fois les clauses construites, le programme écrit un fichier au format **DIMACS CNF** dans le dossier `Dimacs/`. Ce fichier permet de visualiser le probleme SAT genere et de garder une trace standard de l'instance.

Enfin, le fichier DIMACS est relu, les clauses sont envoyees à **Sat4J**, puis le modele trouve est transforme en grille lisible. Si une premiere solution existe, le programme ajoute aussi une clause qui interdit de retrouver exactement cette meme solution, puis relance le solveur pour verifier s'il existe une deuxieme solution.

Variantes traitees

J'ai implemente les trois variantes demandees dans le sujet.

Pour la variante `Basic`, une grille de taille `n x n` utilise `n` lettres. Chaque lettre doit etre presente une seule fois dans chaque ligne et dans chaque colonne. Les indices sur les bords indiquent directement la premiere lettre visible depuis la direction correspondante.

Pour la variante `Easy1`, la grille utilise `n - 1` lettres. Il y a exactement une case vide dans chaque ligne et dans chaque colonne. Les cases vides sont representees par `X` et ne comptent pas pour les indices exterieurs.

Pour la variante `Easy2`, la grille utilise `n - 2` lettres. Cette fois, chaque ligne et chaque colonne contiennent exactement deux cases vides. Comme pour `Easy1`, les `X` sont ignores lorsqu'on cherche la premiere lettre visible depuis un bord.

Choix techniques

Le choix principal a ete de conserver une separation claire entre la construction du probleme SAT et sa resolution. Le programme ne donne pas directement les clauses a Sat4J au moment de leur creation : il genere d'abord un fichier DIMACS, puis recharge ce fichier pour la resolution. Cette approche rend le fonctionnement plus facile a verifier, car le fichier CNF reste disponible dans `Dimacs/`.

J'ai aussi choisi d'afficher beaucoup d'informations pendant l'execution : variante du puzzle, taille de la grille, nombre de variables, nombre de clauses, statistiques sur les clauses et nom du solveur utilise. Cela permet de comprendre ce que le programme fait sans devoir ouvrir le code.

Pour la recherche d'une deuxieme solution, j'ai utilise la methode vue en cours : apres avoir obtenu un modele SAT, je construis une clause contenant la negation de tous les litteraux positifs de cette solution. En ajoutant cette clause, Sat4J ne peut plus retourner exactement le meme modele.

Tests et resultats

J'ai teste le programme avec les instances presentes dans le dossier `Puzzle/` :

- `puzzle.txt`
- `puzzle_easy1.txt`
- `puzzle_easy2.txt`
- `puzzle_test_seconde_solution.txt`

La commande :

```
make test
```

lance automatiquement ces quatre instances.

Les tests montrent que les trois variantes sont bien résolues. Pour chaque puzzle, le programme genere un fichier DIMACS, charge les clauses dans Sat4J et affiche une grille solution.

Pour répondre à la partie du sujet sur la génération automatique, j'ai ajouté une classe `PuzzleGenerator`. Elle construit automatiquement des grilles valides, calcule les indices extérieurs correspondants, puis écrit de nouveaux fichiers de puzzle dans le dossier `Puzzle/`.

La commande :

```
make generate
```

genere plusieurs instances de tailles différentes :

- `generated_basic_4.txt`
- `generated_basic_6.txt`
- `generated_easy1_5.txt`
- `generated_easy2_6.txt`

Ces instances permettent de tester à la fois plusieurs variantes et plusieurs tailles de grille. La commande :

```
make test-generated
```

genere ces instances puis les résout automatiquement avec Sat4J.

L'instance `puzzle_test_seconde_solution.txt` sert à vérifier la recherche d'une deuxième solution. Sur cette instance, le programme trouve bien une première solution, ajoute une clause de blocage, puis trouve une seconde solution différente.

Les variantes avec cases vides fonctionnent également. Dans `Easy1`, chaque ligne et chaque colonne contiennent bien une case `X`. Dans `Easy2`, chaque ligne et chaque colonne contiennent bien deux cases `X`.